Updates

Ansh Bansal

March 2026

1 March 2nd week

Target:

- Designing the environment and simulations.

1.1 before 16th March

- Created the enironment `multiagent_env_stable_baselines.py` Env Description
- Designed the reward functions and implemented.
- Facing problem in evaluating the safe space in 2-D, as Nischal has done it in 1-D for rear and front vehicles but in this case it is difficult to design proper optimization problem to find the safe action set.
- Searching for more methods for neighbourhood vizualisation rather than global. [3]

1.2 16th March

- Shifted matplotlib visualization to PyGame
- Shifted from stable_baselines to RLlib (advantages: faster, distributed system, strong multi-agent support). Created new file `multiagent_env_rllib.py`.
- Looking for a neighborhood visualization method for each agent at every timestep [2]

2 March 3rd week

Target:

- Modifying the environment to make it compatible with RLlib.

2.1 17th March

- Tried replacing stable_baselines with RLlib but getting some error

2.2 18th March

- Successfully replaced the stable_baselines with RLlib `multiagent_env_rllib.py`.
- Made some changes in PyGame visualizations `pygame_visualizations.py`

3 March 4th week

Target:

- Designing an attention based algorithm for neighbourhood management (collision avoidance).

3.1 29th March

- Read *Mix Q-learning for Lane Changing* [1] Summary

3.2 30th March

- Added grids in the visualization.
- Renamed `multiagent_env_rllib` as `multiagent_env_rllib_17032026.py`
- Organized the environment `multiagent_env_rllib_30032026.py`.
- Modified `pygame_visualizations.py` accordingly.
- Tried using implementing attention mechanism.
- Created `local_attention_30031026.py`

3.3 31st March

- Formulated attention mechanism of transformer to apply for each agent. Formulation
- Modified the observation functions in the environment in `multiagent_env_rllib_30032026.py`.
- Modified `local_attention_30031026.py` accordingly.

4 April 1st week

Target:

- Experimenting with the algorithm.
- Making the algorithm more robust.

4.1 1st April

- Wrote the code for PPO algorithm according to the attention mechanism `ppo_multiagent_attention.py`.
- Ran the algorithm on this system for 100 and 200 iterations for validation.
- Overall the rewards were fluctuating, hence not good results.

4.2 2nd April

- Ran the algorithm on this system (CPU) for 100, 1000, and 2000 iterations.

4.3 3rd April

- Tried to run the algorithm on server's GPU but there was some error.

4.4 5th April

- The plot of total rewards for 2000 iterations was fluctuating and rough average was around 0. Need to modify the network architecture.

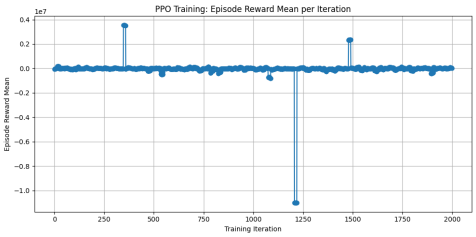

Figure 1: With Attention

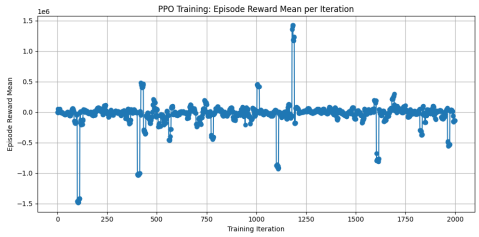

Figure 2: Without Attention

- Tried to run the algorithm on server's GPU but there was some error.

4.5 6th April

- TRIED SOMETHING WITH THE CODE (WASN'T HELPFUL).

5 April 2nd week

Target:

- Designing robust attention based algorithm.

5.1 7th April

- Looked out for different algorithms provided by RLlib.
- Tried Asynchronous Proximal Policy Optimisation (APPO) `appo_multiagent_attention.py`.
- Performed even worse than PPO.

5.2 8th April

- After trying so many things, the code ran on GPU.

5.3 9th April

- Tried changing hyperparameters.
- Sumit suggested to use the entropy `entropy_coeff`. Started performing better.
- Ran an equivalent code without attention `ppo_multiagent_wo_attention.py`

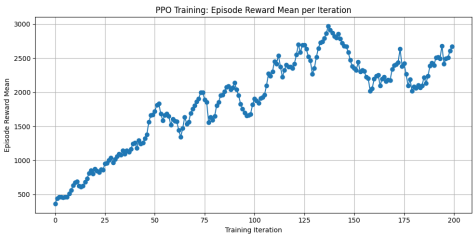

Figure 3: With Attention

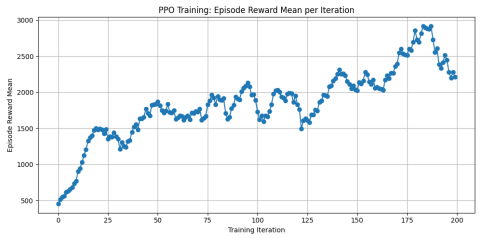

Figure 4: Without Attention

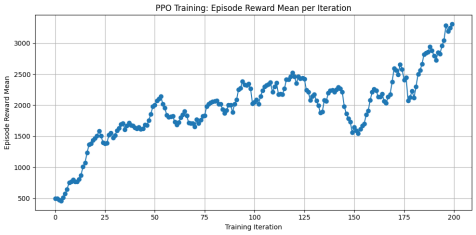

Figure 5: With Attention

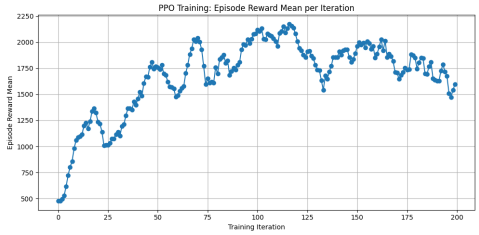

Figure 6: Without Attention

5.4 10th April

- 2000 Iterations with the simulation taking approximately two days to complete.

5.5 11th April

- There was no significant difference between the `ppo_multiagent_wo_attention.py` and `ppo_multiagent_attention.py` .

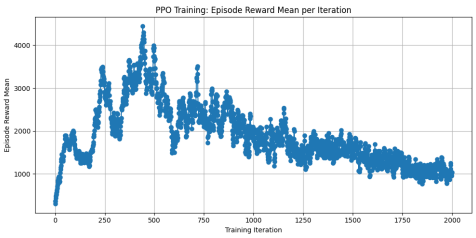

Figure 7: With Attention

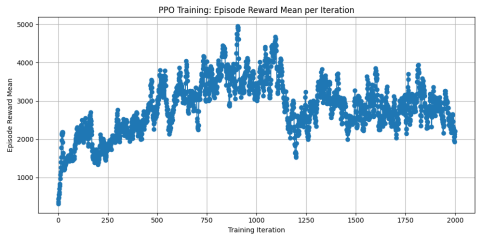

Figure 8: Without Attention

- Ran the code again.

5.6 12th April

- The code was still running but still there was no significant difference to establish the attention mechanism will perform better.
- Modified the PPO to MAPPO `mappo_training_attention.py` and an equivalent code without attention `mappo_training_wo_attention.py`
- Added metrics `Collision Rate` and `Collision Episode` to compare attention wrt w/o attention.

$$collision_rate = collisions/steps$$

$$collision_episode = number\ of\ collsions$$

5.7 13th April

- Comparison of `mappo_training_attention.py` and `mappo_training_wo_attention.py`

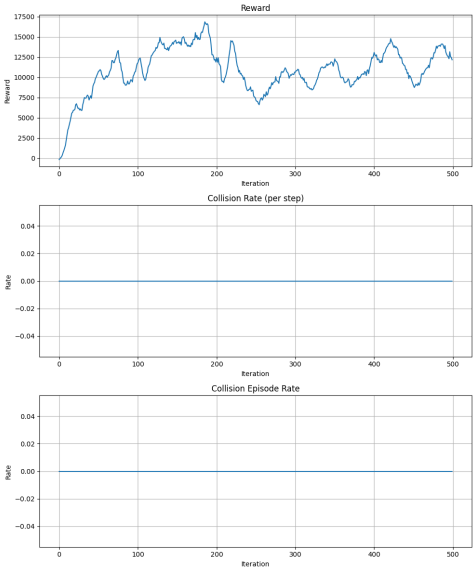

Figure 9: With Attention

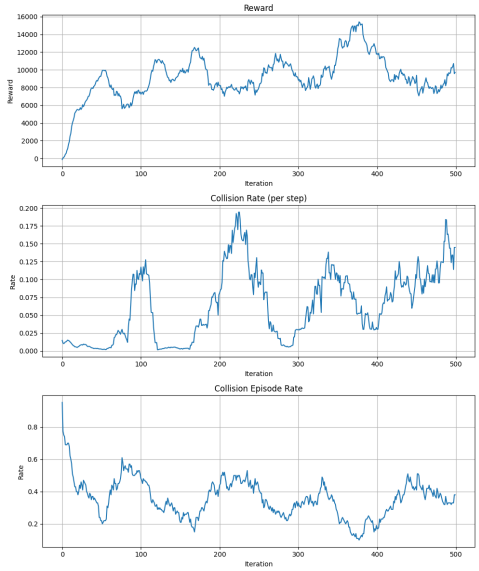

Figure 10: Without Attention

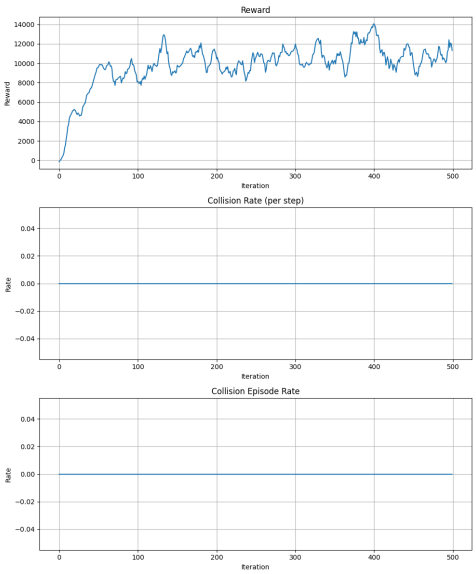

Figure 11: With Attention

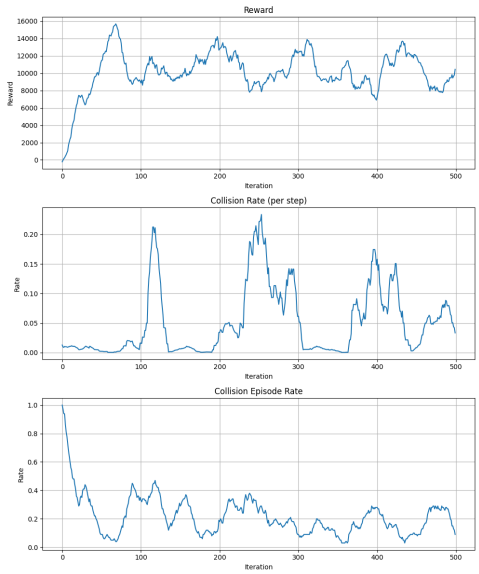

Figure 12: Without Attention

6 April 3rd week

Target:

- Make the attention mechanism more robust, which guaranties collisions will become zero.
- Compare the collision rate, avg velocity, and other relevant metrics in the PPO and MAPPO algorithms.
- Designing the optimization problem for collision avoidance to find the safe action space.

6.1 14th April

References

[1] Xiaojun Bi, Mingjie He, and Yiwen Sun. *Mix Q-learning for Lane Changing: A Collaborative Decision-Making Method in Multi-Agent Deep Reinforcement Learning*. 2025. arXiv: 2406.09755 [cs.AI]. URL: https://arxiv.org/abs/2406.09755.

[2] Edouard Leurent and Jean Mercat. *Social Attention for Autonomous Decision-Making in Dense Traffic*. 2019. arXiv: 1911.12250 [cs.LG]. URL: https://arxiv.org/abs/1911.12250.

[3] Kunpeng Xu, Lifei Chen, and Shengrui Wang. *DRNet: A Decision-Making Method for Autonomous Lane Changingwith Deep Reinforcement Learning*. 2024. arXiv: 2311.01602 [cs.RO]. URL: https://arxiv.org/abs/2311.01602.